

A.S.D:

Chon: Intro

Structure d'un algorithme

Algorithme <nom de l'algorithme>

CONST

nom_de_la_const = valeur

TYPE

Tab ou struct

VAR

nom_var : Type_var.

FONC

nom_des_fonc

PROC

nom_des_proc

DEBUT

{ Corps }

FIN.

Boucle Pour:

Pour <compteur> de <initia> à <finale> pas de

<Trait>

Fin pour

Boucle Selon:

Selon <v>

<v1> : <Trait>

<v2> : <Trait>

...

<vn> : <Trait>

sinon : <Trait>

Fin selon

Type Tableau:

<NomType> = tableau [a_1 .. a_2] de <type de base>

Type intervalle:

type nom_intervalle = borne_inf .. borne_sup

Les enregistrements:

<NomType> = struct

<champ1> : <Type1>

...

<champn> : <Type n>

FIN struct.

Accès: <id-var> : <NomType struct>
<id-var> . <nom_champ>

Tableau:

Tab : Tableau [1..Max] de <nom_Type_struct>

Procédures:

Déclaration:

Procédure <Nom procédure> (<par1> : <Type1> ...)

Var

<variables locales>

Début

<Traitement>

Fin

Appel:

<nom_procedure> (<par1>, <par2>, ...)

Ré: Nom <par1> : <Type1> : pour changer le contenu de <par1> dans le procédure.

Fonction:

Déclaration:

par. formel

Fonction <Nom_Fct> (<par1> : <Type1>, ...) : <Type retour>

Var

<variables locales>

Début

<Traitement>

(<Nom fonction> ← <variable>)

Fin ^{ou rien} retourner()

Appel: var ← nom_fct (<par1>, ...)

par. effectifs

Rekursivité:

Rekursivité directe:

Module A()

DEBUT

...

A()

...

FIN

Rekursivité indirecte:

Module A()

DEBUT

...

B()

...

FIN

Module B()

DEBUT

...

A()

...

FIN

Structure:

Module A()

DEBUT

si < condition d'arrêt > alors

< cas particuliers de récursivité >

si non

retourner (A())

fin si

fin

Fonctions prédéfinies:

- Pos(chaine, sous chaine) : -1 si n'existe pas
- Efface(ch, position départ, longueur)

• Longueur(chaine)

• Concat(ch1, ch2)

• Majuscule(chaine), minuscule(chaine)

• Insérer(chaine, sous chaine, position)

• Sous chaine(chaine, position, longueur)

Les pointeurs:

Déclaration:

id_var pointeur : ^ type de base.

Réservation de la mémoire:

~~allouer~~

allouer(id_var pointeur)

Grâce à la variable pointeur,

id_var pointeur ^

Libération de la mémoire:

Libérer(id_var pointeur)

Rq: Le pointeur particulier Nil indique l'absence d'adresse.

Pointeurs et tableau:

L : ^ Tab

allouer(L)

L[i] : i^{ème} case.

Ch02: Les Listes

1. représentation chaînée:

* Déclaration:

```
noeud = struct  
    Val: Element  
    Suiv: ^noeud  
Fin struct  
ListCh = ^noeud
```

- La liste vide est représentée par le pointeur Nil.
- Si l ≠ liste vide alors l contient l'@ de la tête de la liste

* Use:

- pointeur:
 - ↳ Déclaration: <pointeur_nom>: ListCh
 - ↳ Création: <pointeur_nom>.[^]Suiv ← @olier

* Liste Circulaire:

1. Def: dans la dernière place de la liste, le pointeur nil par un pointeur vers la tête de la liste → utile pour les files.

* Liste doublement chaînée:

- Def: Chaque place de cette liste contient un pointeur vers la place suivante et un pointeur vers la place précédente.

* Déclaration:

```
LD = struct  
    First: ^Noeud  
    Last: ^Noeud  
    Fin struct  
Noeud = struct  
    Pred: ^Noeud  
    Suiv: ^Noeud  
    Fin struct
```

- La première case est précédée par Nil.

CH03 : Pile et File

1. Pile : (LIFO)

a. Opérations autorisées :

- (P) * Est_Vide : Tester si une pile est vide ou non
- (P,d) * Empiler : insérer au sommet un elt
- (P,d) * Dépiler : retirer l'élément qui se trouve au som
- (P,e) * Sommet : retourner l'élément qui a été inséré.

2. Pré-conditions :

- * Dépiler (P) défini ssi Est_vide (P) = faux
- * Sommet (P) défini ssi Est_vide (P) = faux.

3. Représentation des piles

a. Rep. contigue :

Pile = struct

taille : entier

Tob : tableau [1..Max] d'elt.

fin struct.

b. Rep. chaînée :

Cellule = struct

val : Elt

Suiv : ^cellule

fin struct

Pile = ^cellule

II. Files : (FIFO)

1. Opérations autorisées :

- * Est_vide : Tester si la file est vide
- * Enfiler : insérer un élément en queue.
- * Défiler : supprimer le premier elt de file
- * Premier : accéder à la tête

2. Pré-conditions :

- * Défiler (F) défini ssi Est_vide (F) = faux
- * Premier (F) défini ssi Est_vide (F) = faux

3. Rep. des files :

- a. Rep. contigue : (il faut connaître la tête et la queue)

File = struct

FA : tableau [1..Max] d'elt.

t : entier ou 0..Max

q : entier ou 0..Max

fin struct.

b. Rep. chaînée :

Cellule = struct

val : Elément

Suiv : ^cellule

fin struct

File = struct.

T, Q : ^cellule

fin struct.

Où bien : on peut utiliser une liste circulaire pour représenter une file. Dans ce cas on utilise un seul pointeur.

CH01: Les arbres Binaires

I - Les arbres binaires :

1 - Vocabulaire :

- Racine : nœud de niveau zéro (n'a pas de père)
- Nœud : racine (ou sommet) de tout sous-arbre.
- Chaque nœud a 0 ou plusieurs fils.
- Feuille : un nœud qui n'a pas de fils.
- Fils : x est fils de y si x est la racine de sous-arbre y .
- Père : x est le père de y si y est le fils de x .
- Frère : deux nœuds sont frères s'ils ont le même père.
- Ascendants : A est ascendant de B si A est le père de B ou un ascendant de B .
- Descendants : x est desc. de y si x est le fils de y ou un descendant d'un fils de y .
- Branche : tout chemin de la racine à une feuille de l'arbre.
- Hauteur (ou profondeur au niveau) d'un nœud :
 - étant donné x un nœud d'un arbre B ,
 $h(x) = 0$ si x racine de B
 $h(x) = 1 + h(y)$ si y est le père de x

2 - Arbre binaire (type)

- soit vide noté $\emptyset$
- soit de forme : $B = \langle o, B_1, B_2 \rangle$; où
 - B_1 : sous-arbre gauche de la racine de B
 - B_2 : sous-arbre droite de B
 - o : un nœud appelé racine

3 - Opérations sur un arbre binaire :

- Arbre-vide $\rightarrow$ Arbre : crée un arbre vide
- $\langle -, -, - \rangle$: Nœud $\times$ Arbre $\times$ Arbre $\rightarrow$ Arbre :
 crée un nouvel arbre binaire avec le nœud comme racine et les deux Arbres comme sous-arbres gauche et
- g : Arbre $\rightarrow$ Arbre : prend un arbre et rend le sous-arbre gauche de cet arbre.
- d : Arbre $\rightarrow$ Arbre : // sous-arbre droite.
- r : Arbre $\rightarrow$ Nœud : retourne le nœud racine d'un

Contenu : Nœud $\rightarrow$ élément : prend un nœud et retourne l'élément contenu dans ce nœud.

4 - Conditions :

- racine (B_1) def $\forall$ $B_1 \leftrightarrow$ Arbre-vide.
- $g(B_1)$ def $\forall$ $B_1 \leftrightarrow$ Arbre-vide
- $d(B_1)$ def $\forall$ $B_1 \leftrightarrow$ Arbre vide.

5 - Résu :

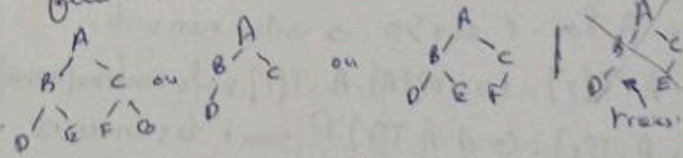
- racine($\langle o, B_1, B_2 \rangle$) = o
- $g(\langle o, B_1, B_2 \rangle) = B_1$; $d(\langle o, B_1, B_2 \rangle) = B_2$.

6 - Mesures :

- taille (arbre-vide) = 0
- taille($\langle o, B_1, B_2 \rangle$) = $1 + \text{taille}(B_1) + \text{taille}(B_2)$
 $\rightarrow$ la taille d'un arbre est le nombre de ses nœuds.
- hauteur d'un nœud
- hauteur ou profondeur d'un arbre B :
 $h(B) = \max \{ h(x) ; x \text{ nœud de } B \}$

2 - Arbres binaires particuliers :

- dégénéré ou filiforme : tous ses nœuds ont au maximum un seul fils.
- Complet : contient 2^h nœuds au niveau h (h en niv 0 ; 2 en niv 1 ; ...)
- plein : tous les niveaux sont remplis sauf le dernier et dans ce cas les feuilles du dernier sont groupées à gauche.



8. Représentation chaînée d'un arbre:

Noeud = struct
 Val : Element
 g, d : ^Noeud
 Fin struct

Type Arbre = ^Noeud

- $A = \text{nil} \Leftrightarrow A = \text{arbre-vide}$
- $A^{\wedge}.val \Leftrightarrow \text{Contenu}(\text{racine}(A))$
- $A^{\wedge}.g \Leftrightarrow g(A) \quad A^{\wedge}.d \Leftrightarrow d(A)$

9. Représentation contiguë:

Le nombre de noeuds de l'arbre est limité et ne doit pas dépasser une valeur Max.

* Rep. indexée:

→ à chaque noeud de l'arbre on associe une valeur ainsi que les indices des deux fils gauche et droit.

Env: Indix = 0

Noeud = struct
 Val : Element
 g, d : 0..Max

Fin struct
 Type ArbTab = tableau [1..Max] de Noeud
 Arbre = struct

Rac : 0..Max

T : ArbTab

Fin struct

- $A.Rac = 0 \Leftrightarrow A = \text{arbre-vide}$
- $A.Rac = r \text{ et } r > 0 \Rightarrow \text{arbre non vide}$
- $A.T[r] = \text{racine}(A), A.T[r].val = \text{Contenu}(\text{racine}(A))$
- $A.T[r].G \text{ et } A.T[r].D$ sont les indices de $g(A)$ et $d(A)$.

* Rep. séquentielle:

Convention:

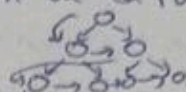
- si noeud nu mérite par i
- fils gauche: $2i$
 - fils droit: $2i+1$

Et aussi un tableau au

10. Parcours d'un A.B:

* Parcours en largeur: (BFS):

→ essayer toujours de visiter le noeud le plus proche de la racine qui n'a pas été visité.



→ on va d'abord visiter la racine puis les noeuds à la profondeur 1, puis 2...

* Toujours en profondeur mais gauche (DFS) (File)

* Implémentation itérative:

F: file q: Arbre.

A: arbre

Enfiler(A) Non vide (file) faire.

Tant que

Defiler(F, q)
 Faire(q.val)
 si $g(q) \neq \text{nil}$ Alors
 Enfiler(file, g(q))
 Fin
 si $d(q) \neq \text{nil}$ Alors
 Enfiler(file, d(q))
 Fin

Fin

Parcours par profondeur main gauche: (DFS)
implémentation par pile

ce chemin part à gauche de la racine et va toujours le plus profond possible à gauche. soit existe 3 orches classiques d'exploration de l'arbre: traite les nœuds avant leurs fils.

- 1- Orche préfixe: père, fils gauche, fils droit.
- 2- infixe: fils gauche, père, fils droit.
 Contraite le nœud entre ses enfants.
- 3- postfixe: fils gauche, fils droit, père.
 Contraite les nœuds après ses fils.

Implémentation:

1- Parcours préfixe:

recurs:

Proc L-Pré (Don A: Arbre):
 Debut
 si (A <> Arbre vide) alors
 Ecrire (A.val)
 L-Pré (g(A))
 L-Pré (d(A))
 Finsi
 Fin.

iteratives:

version 1:

Cellules struct
 val: Arbre
 suiv: ^ cellule
 Fin struct
 Lile = ^ cellule
 Procédure L-Préfixe (Don: A: Arbre)
 var
 L: Lile
 Debut
 Lile - vide(L)
 Tq (A <> nil) ou (non Est vide(L)) faire
 Tq (A <> nil) faire
 Ecrire (A.val)
 Empiler (L, A)
 A ← g(A)
 Fin Tq
 Fin Tq

Depiler (L, A)
 A ← D(A)

FTQ
 Fin

version 2:

Proc L-Préfixe (Don A: Arbre)
 var
 L: Lile
 q: Arbre
 Debut
 q ← A ; Lile - vide(L)
 Tq (q <> nil) faire
 Ecrire (q.val)
 si (d(q) <> nil) Alors Empiler (L, d(q))
 si (g(q) <> nil) Alors q ← g(q)
 sinon depiler (L, q) fsi.
 FTq.

Fin

version 3:

Empiler (L, A)
 Tq (non Est vide(L)) faire.
 Depiler (L, q)
 si (q <> nil) alors
 Ecrire (q.val)
 Empiler (L, d(q))
 si g(q) <> nil Alors q ← g(q)
 sinon depiler (L, q) Fin

Proi.

FTq Fin.

IV. Les arbres binaires de recherche:

Def: tel que v noté $d(v)$
 - des éléments de tous les nœuds du sous arbre gauche de v sont inférieurs ou égaux à l'élément contenu dans v
 - des éléments de tous les nœuds du sous arbre droit de v sont supérieurs à l'élément contenu dans v
 → recherche et tri efficaces.

1. Recherche

Proc recherche (Don x : Elém, A : Arbre): booléen

Debut
 Si $A = \text{nil}$ alors Recherche ← faux
 Sinon si $x = A^{\text{val}}$ alors rech ← vrai
 sinon si $x < A^{\text{val}}$ alors Rech ← recherche($x, g(A)$)
 sinon Recherche($x, d(A)$)
 Fin si
 Fin

2. Adjontion aux feuilles:

Après on compare l'élément au contenu de la racine pour savoir si l'ajout se fait dans s.o.g ou s.o.d.

Algorithme:

Procédure Ajouter (Don x : Elém: Don des A : Arbre)

Debut
 si $A = \text{nil}$ alors
 Créer (A)
 $A^{\text{val}} \leftarrow x$
 $A^{\text{g}} \leftarrow \text{nil}$
 $A^{\text{d}} \leftarrow \text{nil}$
 sinon si $x < A^{\text{val}}$ alors Ajouter(x, A^{g})
 sinon Ajouter(x, A^{d})
 Fin si
 Fin

3. Adjontion à la racine:

il faut tout d'abord faire la coupe avant l'ajout.

Proc Coupe (Don x : Elém, Don des A, G, D : Arbre)

Debut
 si $A = \text{nil}$ alors
 $G \leftarrow \text{nil}$
 $D \leftarrow \text{nil}$
 sinon
 si $A^{\text{val}} > x$ alors
 $D \leftarrow A$
 Coupe($x, A^{\text{g}}, G, D^{\text{g}}$)
 Fin si
 Fin

si non
 $G \leftarrow A$
 Coupe($x, A^{\text{d}}, G^{\text{d}}, D$)
 Fin si
 Fin

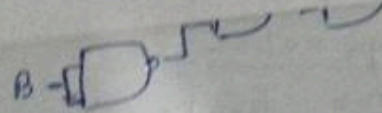
Proc AjouterRac (Don x : Elém, Don des A : Arbre)

Var r : Arbre
 Debut
 Créer(r)
 $r^{\text{val}} \leftarrow x$
 Coupe($x, A, r^{\text{g}}, r^{\text{d}}$)
 $A \leftarrow r$
 Fin

4. Suppression d'un élément:

Proc Supmax (Don Max: Elém, Don des A : Arbre)
 q : Arbre

Debut
 Si $A^{\text{d}} = \text{nil}$ alors
 $q \leftarrow A$
 $\text{Max} \leftarrow A^{\text{val}}$
 $A \leftarrow A^{\text{g}}$
 Libérer(q)
 sinon
 Supmax(Max, A^{d})
 Fin si
 Fin



On élimine la variable qui change dans l'arbre.

Pa 6 Supprimerarb (Den X: Elet Den RSA: A: A: he)

Max: Elet

Output
si A est nil alors
si $X < A^{\wedge}.val$ alors

Supprimerb(X, $A^{\wedge}.g$)

sinon si $X > A^{\wedge}.val$

Supprimerarb(X, $A^{\wedge}.d$)

sinon
si $A^{\wedge}.d = \text{nil}$ alors

$A \leftarrow A^{\wedge}.g$

sinon si $A^{\wedge}.g = \text{nil}$ alors

$A \leftarrow A^{\wedge}.d$

sinon

supmax(Max, $A^{\wedge}.g$)

$A^{\wedge}.val \leftarrow \text{Max}$

Fin

Fin

Fin

Fin